



Web Technologies

WET2VO

Teil 11: REST APIs & AJAX

Medientechnik und -design | SS 2026

REST APIs

Daten am Server strukturiert verfügbar machen

Web Services



Maschine-zu-Maschine-Kommunikation

Backend <-> Frontend

z.B. XML-RPC, JSON-RPC, SOAP, oder **REST**

REST

Representational State Transfer

Ein Softwarearchitekturstil
zur Erstellung von Webservices

HTTP(S)-Requests werden an einen eindeutigen URI abgesetzt.
z.B.: `GET https://api.foo.com/users`





6 Prinzipien von REST

Client-Server

Trennung von Aufgaben.
Server: Stellt den Dienst bereit.
Client: Konsumiert den Dienst.

Zustandslosigkeit

Die Antwort muss alle Daten enthalten, um die Anfrage zu beantworten. Es wird kein Zustand gespeichert.

Caching

Antworten sollen zwischengespeichert werden können.

Einheitliche Schnittstelle

Eindeutig identifizierbar (URI),
verschiedene Formate (JSON/XML),
selbstbeschreibende Nachrichten, HATEOAS-Prinzip.

Mehrschichtiges System

Der Client erkennt nicht, ob eine direkte Verbindung herrscht, oder ob Systeme dazwischen sind.

Code on Demand (Opt.)

Rest-Schnittstellen können optional Code mitliefern.

Aufbau einer REST API

Die Idee dahinter



REST & HTTP-Methoden

GET

Abfragen von Daten, z.B.

- Alle User*innen
- User mit einer ID

GET `https://api.foo.com/users`

GET `https://api.foo.com/users/3`

DELETE

Daten löschen, z.B.

- User*in mit bestimmter ID löschen.

DELETE `https://api.foo.com/users/3`

POST

Erstellen von Daten, z.B.

- User*in anlegen

POST `https://api.foo.com/users`

HEAD

Informationen über Daten erhalten, z.B.

- Größe der User*innenliste

HEAD `https://api.foo.com/users`

PUT

Aktualisieren von Daten, z.B.

- User*in mit bestimmter ID updaten.

PUT `https://api.foo.com/users/3`

OPTIONS, TRACE, CONNECT

Weitere HTTP-Methoden sind ebenfalls möglich.



Das **Datenformat** bekanntgeben

Client

Setzt beim **Request** den **Accept-Header**. Dies teilt mit, welche Formate akzeptiert werden.

Accept: text/plain,application/json

Server

Setzt bei der **Response** den **Content-Type-Header**. Dieser teilt mit, welchen Typs die Antwort ist.

Content-Type: application/json



Wichtige Statuscodes

Jede Response enthält einen Statuscode.

2xx: Erfolgreich	4xx: Client-Fehler	5xx: Server-Fehler
200 OK	400 BAD REQUEST	500 INTERNAL SERVER ERROR
201 CREATED	401 UNAUTHORIZED	501 NOT IMPLEMENTED
204 NO CONTENT	403 FORBIDDEN	
	404 NOT FOUND	
	406 NOT ACCEPTABLE	



Datenformate der REST-Response

XML

Mittels `XMLWriter`
oder `DOM`.

JSON

Mittels
`json_encode()`.

HTML

Fragmente zum
direkten Einfügen
auf der Clientseite.

TEXT

Zeilenweise oder
als CSV.

XML-Response



GET <https://api.foo.com/users>

```
<users size="3">
  <link rel="self" href="/users" />
  <user id="1">
    <username>jdoe</username>
    <realname>John Doe</realname>
    <link rel="self" href="/users/1" />
  </user>
  <user id="2">
    <username>jane789</username>
    <realname>Jane Doe</realname>
    <link rel="self" href="/users/2" />
  </user>
  <user id="3">
    <username>jimmy</username>
    <realname>Jim Doe</realname>
    <link rel="self" href="/users/3" />
  </user>
</users>
```

JSON-Response



GET https://api.foo.com/users

```
{
  "size": 3,
  "link": "/users",
  "users": [
    {
      "id": 1,
      "username": "jdoe",
      "realname": "John Doe",
      "link": "/users/1"
    },
    {
      "id": 2,
      "username": "jane789",
      "realname": "Jane Doe",
      "link": "/users/2"
    },
    {
      "id": 3,
      "username": "jimmy",
      "realname": "Jim Doe",
      "link": "/users/3"
    }
  ]
}
```

HTML-Response



GET https://api.foo.com/users

```
<div>
  <div class="user" id="user1">
    <p class="username">jdoe</p>
    <p class="realname">John Doe</p>
  </div>
  <div class="user" id="user2">
    <p class="username">jane789</p>
    <p class="realname">Jane Doe</p>
  </div>
  <div class="user" id="user3">
    <p class="username">jimmy</p>
    <p class="realname">Jim Doe</p>
  </div>
</div>
```

Text-Response



```
GET https://api.foo.com/users
```

```
1;jdoe;John Doe;/users/1
```

```
2;jane789;Jane Doe;/users/2
```

```
3;jimmy;Jim Doe;/users/3
```

Eine REST API in PHP implementieren

GET und POST Requests, JSON Responses
Mit dem fhoee/router-skeleton

4 Routen anlegen



```
$router→get("/users", function () {  
    $userManager = new UserManager();  
    $userManager→listUsers();  
});  
  
$router→get("/users/{$id}", function ($id) {  
    $userManager = new UserManager();  
    $userManager→listUser($id);  
});  
  
$router→post("/users", function () {  
    $userManager = new UserManager();  
    $userManager→addUser();  
});  
  
$router→get("/adduser", function () use ($latte) {  
    $latte→render("adduser.latte");  
});
```

showResponse()



```
private function showResponse(int $statusCode, ?array $content = null): void
{
    $acceptHeader = $_SERVER["HTTP_ACCEPT"] ?? "";

    if (str_contains($acceptHeader, $this->contentType) ||
        str_contains($acceptHeader, "/*/*")) {
        http_response_code($statusCode);
        header("Content-Type: {$this->contentType}");
        if (isset($content)) {
            echo json_encode($content, JSON_PRETTY_PRINT |
                            JSON_UNESCAPED_SLASHES |
                            JSON_UNESCAPED_UNICODE);
        }
    } else {
        http_response_code(406);
    }
}
```

application/json

Wildcard (alles)

listUsers()



```
public function listUsers(): void
{
    $query = "SELECT * FROM rest_api_example.user";

    $statement = $this->pdo->query($query);
    $users = $statement->fetchAll();

    $jsonResponse = [
        "size" => count($users),
        "link" => $this->link
    ];

    foreach ($users as $user) {
        $newUser = $user;
        $newUser["link"] = "$this->link/{$user["id"]}";
        $jsonResponse["users"][] = $newUser;
    }
    $this->showResponse(200, $jsonResponse);
}
```

/users

GET <https://api.foo.com/users>

```
{
  "size": 3,
  "link": "/users",
  "users": [
    {
      "id": 1,
      "username": "jdoe",
      "realname": "John Doe",
      "link": "/users/1"
    },
    {
      "id": 2,
      "username": "jane789",
      "realname": "Jane Doe",
      "link": "/users/2"
    },
    {
      "id": 3,
      "username": "jimmy",
      "realname": "Jim Doe",
      "link": "/users/3"
    }
  ]
}
```

listUser(\$id)



```
public function listUser(int|string $id): void
{
    if (is_string($id) && !is_numeric($id)) {
        $this->showResponse(400, ["error" => "ID has to be a number"]);
        return;
    }

    $query = "SELECT * FROM rest_api_example.user WHERE id = :id";

    $statement = $this->pdo->prepare($query);
    $params = [":id" => $id];
    $statement->execute($params);
    $users = $statement->fetchAll();

    if (count($users) === 1) {
        $jsonResponse = $users[0];
        $jsonResponse["link"] = "$this->link/$id";
        $this->showResponse(200, $jsonResponse);
    } else {
        $this->showResponse(404);
    }
}
```

GET <https://api.foo.com/users/1>

```
{
  "id": 1,
  "username": "jdoe",
  "realname": "John Doe",
  "link": "/users/1"
}
```

addUser()



```
public function addUser(): void
{
    $query = "INSERT into rest_api_example.user
              SET username = :username, realname = :realname";

    $statement = $this->pdo->prepare($query);
    $params = [":username" => $_POST["username"],
               ":realname" => $_POST["realname"]];
    $success = $statement->execute($params);

    if ($success) {
        $this->showResponse(201, [
            "message" => "User successfully created",
            "link" => "$this->link/{ $this->pdo->lastInsertId() }",
        ]);
    } else {
        $this->showResponse(500, [
            "error" => "An error occurred while creating the user"
        ]);
    }
}
```

POST <https://api.foo.com/users>

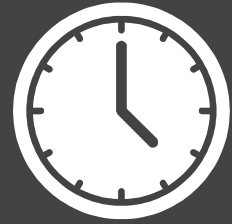
Minimaler Output, nur Statuscode 201 und eine kurze Message.

Es könnte aber z.B. der neu erstellte Eintrag angezeigt werden.

AJAX

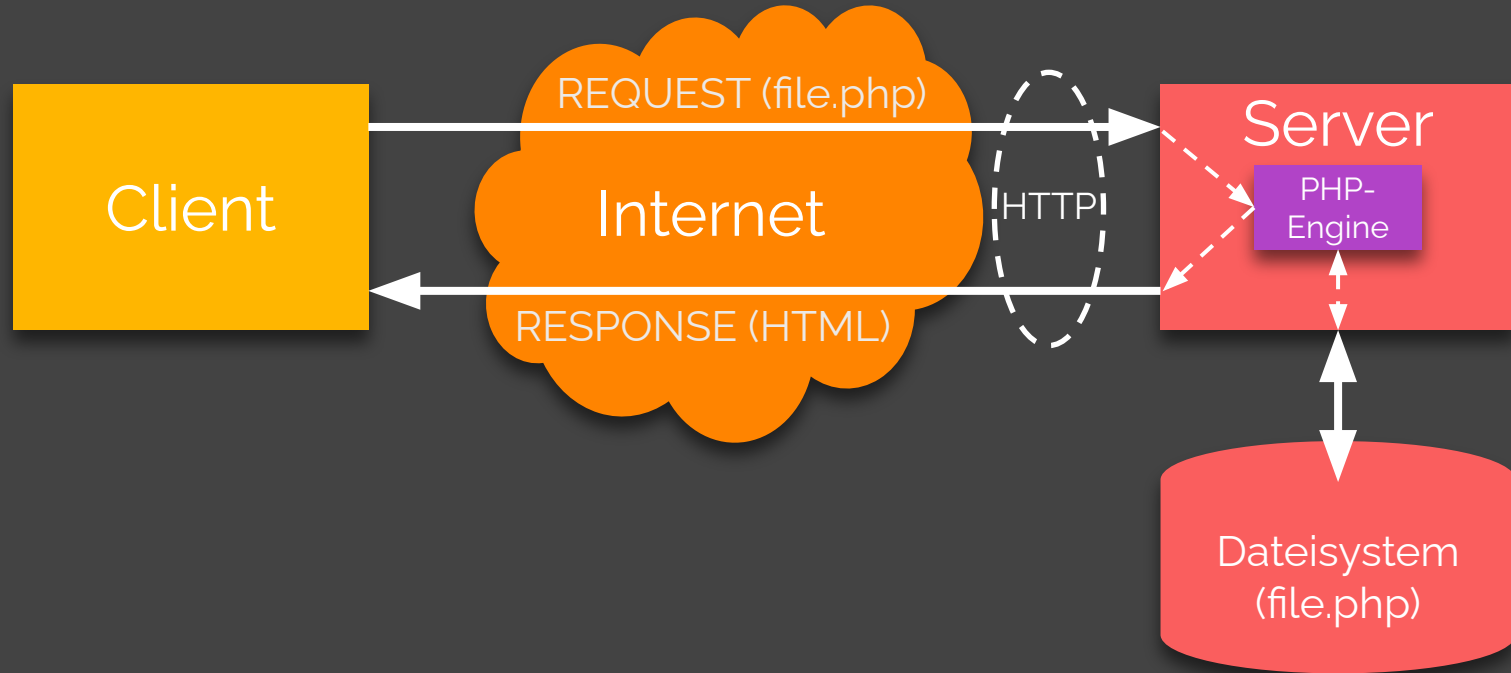
Asynchronous JavaScript and XML

Synchron/ Asynchron

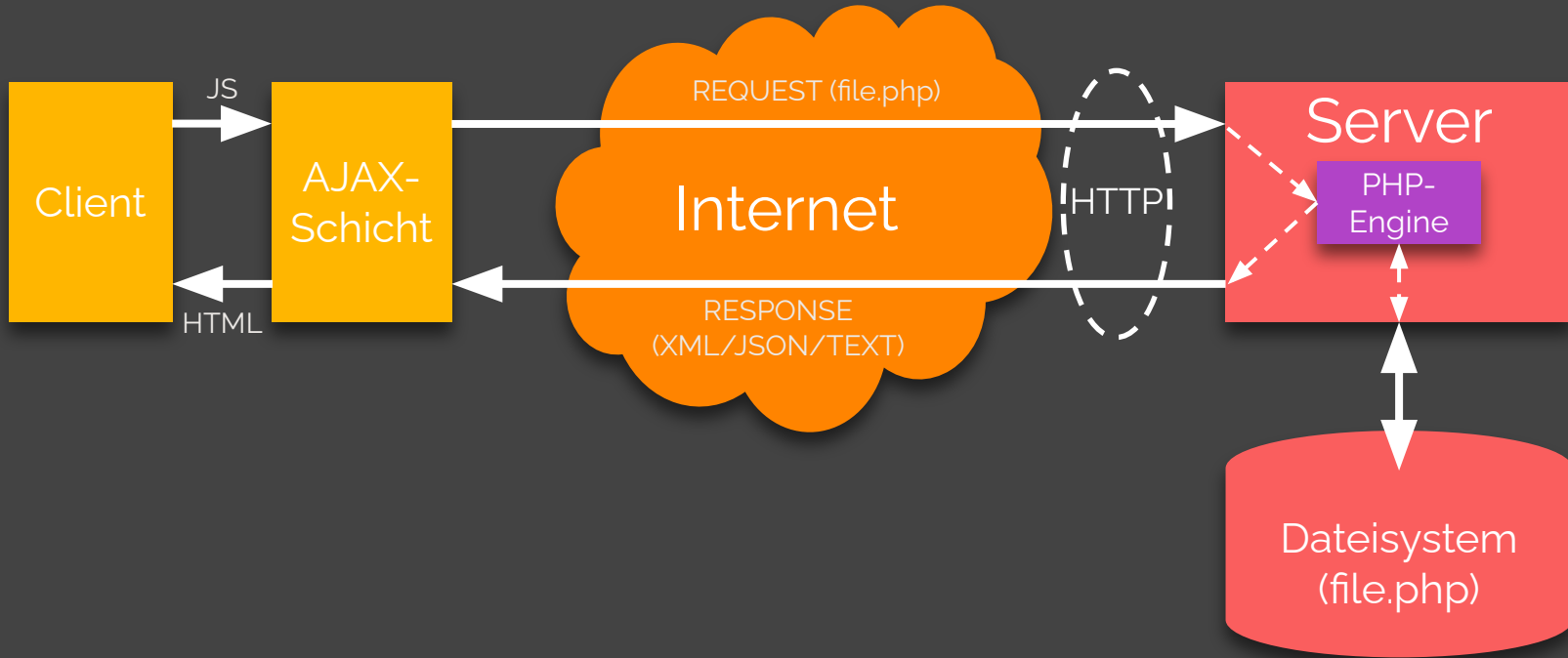


Zeitlich gestaffelte vs.
quasi parallele Abläufe

Klassischer Web-Ablauf



AJAX Ablauf



AJAX mit Fetch

JavaScripts AJAX-Schicht



Die **Fetch** API

Die Fetch API lädt **asynchron** Daten von einem Server und liefert ein **Promise** – einen Platzhalter für ein Ergebnis, das erst später eintrifft (hier das Response-Objekt mit der Antwort des Servers).

Details folgen in Web Architectures (WAR3VO), wir brauchen zunächst nur die 3 Bausteine:

async

Eine Funktion, die auf **Daten aus dem Netzwerk** wartet, wird mit dem Schlüsselwort **async** markiert.

await

Ein Aufruf, dessen **Ergebnis abgewartet** werden muss, wird mit **await** versehen. Das Script „pausiert“ an dieser Stelle, bis das Ergebnis vorliegt, und liefert es dann direkt zurück.

try/catch

Tritt während des Requests ein **Fehler** auf, wird er im **catch**-Block abgefangen.

Prinzipieller Fetch-Ablauf



```
async function loadData() {  
  try {  
    const response = await fetch("daten.php");  
  
    // Antwort enthält HTTP Status zwischen 200 und 299  
    if (response.ok) {  
      const data = await response.json();  
      // data weiterverarbeiten ...  
    } else {  
      // HTTP-Fehler (z. B. 404, 500) behandeln  
    }  
  } catch (error) {  
    // Netzwerkfehler behandeln (Server nicht erreichbar )  
  }  
}
```

Beispiel: Textdaten



Anfrage an eine PHP-Datei (API), die das Ergebnis als Textdaten liefert.
Dieses wird mit JavaScript direkt in die Seite eingebaut. **Kein Reload!**

HTML-Datei



```
<head>
  <script src="ajax.js" defer></script>
  ...
</head>
<body>
  <h1>Show the Capital</h1>
  <p>The following example retrieves the capital of the selected state asynchronously
    and displays it in the heading.</p>
  <h2 id="capital">Select a state to have it replaced with its capital</h2>
  <form action="#">
    <label for="state">Please select a state:</label>
    <select id="state">
      <option>Please select</option>
      <option>Burgenland</option>
      <option>Carinthia</option>
      <option>Lower Austria</option>
      <option>Salzburg</option>
      <option>Styria</option>
      <option>Tyrol</option>
      <option>Upper Austria</option>
      <option>Vorarlberg</option>
      <option>Vienna</option>
    </select>
  </form>
</body>
```

JavaScript / AJAX



```
const select = document.getElementById("state");  
select.addEventListener("change", sendRequest);
```

```
async function sendRequest(event) {  
  const index = event.target.selectedIndex;  
  const capital = document.getElementById("capital");  
  try {  
    const response = await fetch("states.php?index=" + index, {  
      headers: {"Accept": "text/plain"}  
    });  
    if (response.ok) {  
      capital.textContent = await response.text();  
    } else {  
      capital.textContent = "A problem occurred with requesting the selected capital.";  
    }  
  } catch (error) {  
    capital.textContent = "The server could not be reached.";  
  }  
}
```

PHP-Datei / API



```
$capitals = [  
    "Please select a state",  
    "Eisenstadt",  
    "Klagenfurt",  
    "Sankt Pölten",  
    "Salzburg",  
    "Graz",  
    "Innsbruck",  
    "Linz",  
    "Bregenz",  
    "Vienna",  
];  
  
header("Content-Type: text/plain");  
if (!isset($_GET["index"])) {  
    http_response_code(400);  
} elseif (array_key_exists($_GET["index"], $capitals)) {  
    http_response_code(200);  
    echo $capitals[$_GET["index"]];  
} else {  
    http_response_code(404);  
}
```

Beispiel: JSON-Daten



Search Suggest: HTML-Datei mit Suchfeld,
bei Tastendruck, Suche im Datenbestand auf
Übereinstimmung, Rückgabe als
JSON-Struktur. Erstellen einer Liste mit DOM.

HTML-Datei



```
<!DOCTYPE html>
<html lang="en">
  <head>
    <title>AJAX &ndash; Search Suggest</title>
    <meta charset="utf-8">
    <link rel="stylesheet" href="styles.css">
    <script src="ajaxsearch.js" defer></script>
  </head>
  <body>
    <h1>AJAX &ndash; Search Suggest</h1>
    <form id="form" method="get" action="output.php" autocomplete="off">
      <div>
        <label for="search">Search:</label>
        <input type="search" id="search" name="search">
        <button>Search</button>
      </div>
      <div id="suggestions"></div>
    </form>
  </body>
</html>
```

JavaScript / AJAX



```
const search = document.getElementById("search");
search.addEventListener("keyup", sendRequest);

async function sendRequest() {
  try {
    const response = await fetch("search.php?search=" + str, {
      headers: {"Accept": "application/json"}
    });
    const data = await response.json();
    handleResponse(data);
  } catch (error) {
    console.error("Request failed:", error);
  }
}
```

JavaScript / AJAX



```
function handleResponse(data) {  
  const suggestDiv = document.getElementById("suggestions");  
  suggestDiv.innerHTML = "";  
  if (data.count > 0) {  
    for (let i = 0; i < data.count; i++) {  
      const entry = document.createElement("div");  
      entry.addEventListener("mouseover", function () {  
        this.classList.add("suggestLinkOver");  
      });  
      entry.addEventListener("mouseout", function () {  
        this.classList.remove("suggestLinkOver");  
      });  
      entry.addEventListener("click", function () {  
        search.value = this.textContent;  
        suggestDiv.innerHTML = "";  
      });  
      entry.classList.add("suggestLink");  
      entry.textContent = data.words[i];  
      suggestDiv.appendChild(entry);  
    }  
  }  
}
```

PHP-Datei / API



search.php (wird per AJAX aufgerufen)

```
require_once("JSONData/SearchData.php");

use JSONData\SearchData;

$data = new SearchData();

header("Content-Type: application/json");
if (isset($_GET["search"])) {
    http_response_code(200);
    echo $data->search($_GET["search"]);
}
else {
    http_response_code(400);
}
```

PHP-Datei / API



```
class SearchData
{
    private(set) $titles;

    public function __construct()
    {
        $this->titles = [
            "Die FH", "WET2V0", "FH", "Hagenberg", "Datenbanken", "Medientechnik und -design", "MTD25", "MTD",
            "PHP", "JavaScript", "Österreich", "Bla", "Gut/Böse",
        ];
    }

    public function search(string $searchString): string
    {
        if ($searchString ≡ "") {
            $jsonResponse["words"] = [];
            foreach ($this->titles as $entry) {
                if (mb_stripos($entry, $searchString, 0, "UTF-8") ≡ false) {
                    $jsonResponse["words"][] = $entry;
                }
            }
            sort($jsonResponse["words"]);
        }
        $jsonResponse["count"] = count($jsonResponse["words"]);
        return json_encode($jsonResponse);
    }
}
```

Ergebnis bei `$searchString = MT`

```
{
    "words": ["MTD", "MTD25"],
    "count": 2
};
```

Beispiel: POST-Daten



Formular in JavaScript mit der Methode **POST** absenden und Ergebnis ohne Reload anzeigen.

POST-Daten **manuell**



```
fetch("details.php", {  
  method: "POST",  
  headers: { "Content-Type": "application/x-www-form-urlencoded" },  
  body: "foo=bar&test=something",  
});
```

Aber was tun bei vielen Einträgen oder Dateien?



FormData Objekt als Payload

Kompletter Formularinhalt

```
const form =  
document.querySelector("form");  
const formData = new FormData(form);  
Bei fetch(): body: formData;
```

Bestimmte inhalte

```
const formData = new FormData();  
formData.append("foo", "bar");  
formData.append("test", "something");
```

HTML-Datei



```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <title>AJAX POST Request with FormData</title>
    <script src="postform.js" defer></script>
  </head>
  <body>
    <h1>AJAX POST Request with FormData</h1>
    <form action="#" method="post" id="registerForm">
      <label for="user">Name:</label>
      <input type="text" id="user" name="user">
      <label for="pass">Password:</label>
      <input type="password" id="pass" name="pass">
      <button type="submit">Register</button>
    </form>
    <ul id="loginDetails"></ul>
  </body>
</html>
```

JavaScript / AJAX



```
document.getElementById("registerForm").addEventListener("submit", sendRequest);

async function sendRequest(event) {
  event.preventDefault();

  const formData = new FormData(event.target);
  const list = document.getElementById("loginDetails");

  try {
    const response = await fetch("details.php", {
      method: "POST",
      headers: {"Accept": "text/plain"},
      body: formData
    });

    // Verarbeitung (alternativ auch in eigener Funktion)
  }
}
```

JavaScript / AJAX



```
if (response.ok) {  
  const text = await response.text();  
  const entries = text.split("\n");  
  
  for (let i = 0; i < entries.length; i++) {  
    const item = document.createElement("li");  
    item.textContent = entries[i];  
    list.appendChild(item);  
  }  
} else {  
  const item = document.createElement("li");  
  item.textContent = "A problem occurred with the request!";  
  list.appendChild(item);  
}  
} catch (error) {  
  const item = document.createElement("li");  
  item.textContent = "The server could not be reached.";  
  list.appendChild(item);  
}
```

PHP-Datei / API



```
header("Content-Type: text/plain");

if (!empty($_POST)) {
    $lines = [];
    foreach ($_POST as $key => $value) {
        $lines[] = "$key: $value";
    }
    $output = implode(PHP_EOL, $lines);

    http_response_code(200);
    echo $output;
} else {
    http_response_code(400);
}
```

Old School: XMLHttpRequest



```
const request = new XMLHttpRequest();
```

```
function sendRequest(event) {  
  request.open("GET", "states.php?index=" + event.target.selectedIndex, true);  
  request.addEventListener("load", handleResponse);  
  request.send();  
}
```

```
function handleResponse(event) {  
  if (request.status === 200) {  
    document.getElementById("capital").textContent = request.responseText;  
  } else {  
    document.getElementById("capital").textContent = "A problem occurred with  
    requesting the selected capital.";  
  }  
}
```

Ausblick: Promises



async/await mit ES2017 eingeführt. Lesbarere Variante der ursprünglichen Schreibweise:

```
fetch(url)
  .then(response => response.json())
  .then(data => {
    // data verarbeiten
  })
  .catch(error => {
    // Fehler verarbeiten
  });
```

Mehr zu Promises in Web Architectures (WAR3VO)

REST am Server konsumieren

Der Server wird zum Client



HTTP-Clients nach PSR-18 Standard

`guzzlehttp/guzzle`

Bekanntester und am
weitesten verbreiteter
HTTP Client für PHP.

`symfony/httpclient`

PHP-Client des
Symfony-Frameworks.

`kriswallsmith/buzz`

Kleinsten HTTP-Client mit
Fokus auf Performance.

Guzzle

HTTP-Client Bibliothek

Version 7.11.1 (07. Juni 2026)

<https://docs.guzzlephp.org/>





Installation

```
$ composer require guzzlehttp/guzzle
```

Einbindung mittels Composer-Autoloader:

```
require "vendor/autoload.php";
```

Client -Objekt erstellen



Allgemein:

```
use GuzzleHttp\Client;
```

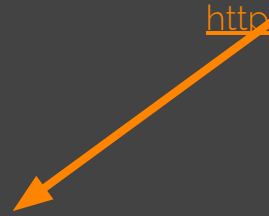
```
$client = new Client();
```

Mit Basepath:

```
$client = new Client([  
    "base_uri" => "https://httpbin.org"  
]);
```

`base_uri` sinnvoll, wenn
mehrere Requests an denselben
URL, z.B.

<https://httpbin.org/post> und
<https://httpbin.org/get>



Wird dann vor jeden Request-URL gehängt.

Request senden



Allgemein:

```
request($method, $uri, $options)
```

Shorthands:

- `get($uri, $options)`
- `post($uri, $options)`
- `put($uri, $options)`
- `delete($uri, $options)`
- `head($uri, $options)`
- `options($uri, $options)`
- `patch($uri, $options)`

Requests mit Parametern



```
$client = new Client([
    "base_uri" => "https://httpbin.org",
]);

$response = $client->get("/get", [
    "query" => ["foo" => "bar"],
]);

$postResponse = $client->post("/post", [
    "form_params" => ["foo" => "bar"],
]);
```

Response verarbeiten



```
echo <<<GETRESPONSE
<p><strong>The GET Response:</strong><br>
{$getResponse→getStatusCode()}<br>
{$getResponse→getReasonPhrase()}<br>
{$getResponse→getHeader("Content-Type")[0]}</p>
<pre>{$getResponse→getBody()→getContents()}</pre>
GETRESPONSE;
```

```
echo <<<POSTRESPONSE
<p><strong>The POST Response:</strong><br>
{$postResponse→getStatusCode()}<br>
{$postResponse→getReasonPhrase()}<br>
{$postResponse→getHeader("Content-Type")[0]}</p>
<pre>{$postResponse→getBody()→getContents()}</pre>
POSTRESPONSE;
```

Fehler behandeln



```
try {  
    $getResponse = $client→get("/someurlthatdoesntexist", [  
        "query" ⇒ ["foo" ⇒ "bar"],  
    ]);  
} catch (ClientException $e) {  
    $getResponse = $e→getResponse();  
}
```

Für 400er Fehler.
500er: `ServerException`
Übergeordnet: `GuzzleException`

```
try {  
    $postResponse = $client→post("/someurlthatdoesntexist", [  
        "form_params" ⇒ ["foo" ⇒ "bar"],  
    ]);  
} catch (ClientException $e) {  
    $postResponse = $e→getResponse();  
}
```



Nächstes Mal in WET2VO

Microframeworks & Testing

Credits



- S. 49: Guzzle Logo: <https://packagist.org/packages/guzzlehttp/guzzle>